DNS + HTTPS Deployment

A Practical Learning Guide with Real Screenshots

Chapter 1 — What Is DNS?

DNS (Domain Name System) translates human-readable domain names like sidhubote.in into IP addresses like 18.216.74.182 that computers use to locate servers. Without DNS, you'd memorise raw IPs for every website.

Analogy: DNS is the internet's phonebook. You look up a name, it gives you the address.

The DNS Resolution Journey

When you type www.sidhubote.in and hit Enter, this happens in under 200ms:

Step	Player	What Happens
1	Your Browser	Checks its own cache. Hit → done. Miss → continue.
2	Your OS	Checks local DNS cache. Hit → done. Miss → continue.
3	Recursive Resolver	Your ISP's DNS server receives the query and does the work.
4	Root Name Server	Resolver asks root: 'Who handles .in?' Gets TLD server address.
5	TLD Name Server (.in)	Resolver asks: 'Who handles sidhubote.in?' Gets authoritative NS.
6	Authoritative NS	Resolver asks: 'What IP is sidhubote.in?' Gets 18.216.74.182.
7	Answer	IP returned to browser. Browser connects to EC2. Page loads!

Key DNS Record Types We Used in This Lab

Record	Purpose	Your Value
A	Maps domain → IPv4 address	sidhubote.in → 18.216.74.182
A (www)	Maps www subdomain → IPv4	www.sidhubote.in → 18.216.74.182
NS	Delegates zone to name servers	GoDaddy's default NS servers
TXT	SPF, verification, DKIM records	Not configured in this lab
CAA	Restricts which CAs can issue certs	Implicitly Let's Encrypt

Chapter 2 — Practical Lab: Hosting sidhubote.in on AWS EC2

This chapter walks through every step of the lab, with actual terminal screenshots. The goal: deploy a portfolio website on a custom domain with HTTPS using Nginx, GoDaddy DNS, and Let's Encrypt.

🔗 **Lab Stack:** AWS EC2 (Ubuntu 24.04 LTS) • Nginx 1.24 • Domain: sidhubote.in (GoDaddy) • Let's Encrypt SSL • Public IP: 18.216.74.182

Step 1 — SSH into the EC2 Instance

Connect to the server using the .pem private key downloaded when creating the EC2 instance:

```
ssh -i Downloads/sid-ansible.pem ubuntu@18.216.74.182
```

On first connection, type yes to accept the host fingerprint. You'll land on Ubuntu 24.04 LTS.

Successful SSH login to EC2 — Ubuntu 24.04 LTS, IP 172.31.39.53 (private), 18.216.74.182 (public)

```
sidhant@bote:~$ ssh -i Downloads/sid-ansible.pem ubuntu@18.216.74.182
The authenticity of host '18.216.74.182 (18.216.74.182)' can't be established.
ED25519 key fingerprint is SHA256:HYvJeW3SNUnR8/f/7OGFc4AZ1gJQmiH4ud+tz5J8fsI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.216.74.182' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Mar 18 03:39:40 UTC 2026

  System load:  0.12               Temperature:           -273.1 C
  Usage of /:   7.6% of 23.17GB    Processes:             114
  Memory usage: 11%                Users logged in:       0
  Swap usage:   0%                 IPv4 address for ens5: 172.31.39.53

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status



The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-39-53:~$
```

Note: The private IP (172.31.39.53) is internal to AWS. The public IP (18.216.74.182) is what the internet sees and what your DNS A record points to.

Step 2 — Update Packages

Always update the package list before installing anything new:

```
sudo apt update
```

sudo apt update — fetching latest package lists from Ubuntu repos

```
ubuntu@ip-172-31-39-53:~$ sudo apt update
Hit:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:6 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:11 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
```

Step 3 — Install Nginx

Install Nginx, the web server that will serve your website:

```
sudo apt install nginx -y
```

Nginx installation — 2 packages installed (nginx + nginx-common), ~565 KB

```
ubuntu@ip-172-31-39-53:~$ sudo apt install nginx -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 11 not upgraded.
Need to get 565 kB of archives.
After this operation, 1596 kB of additional disk space will be used.
Get:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx-common all 1.24.0-2ubuntu7.6 [43.5 kB]
Get:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx amd64 1.24.0-2ubuntu7.6 [521 kB]
Fetched 565 kB in 0s (25.0 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 72566 files and directories currently installed.)
```

Verify the installation succeeded:

```
nginx -v
# nginx version: nginx/1.24.0 (Ubuntu)
```

Nginx version confirmed: 1.24.0

```
ubuntu@ip-172-31-39-53:~$ nginx -v
nginx version: nginx/1.24.0 (Ubuntu)
ubuntu@ip-172-31-39-53:~$
```

Step 4 — Deploy Your Website

Clone your website from GitHub and move it to Nginx's web root directory:

```
git clone https://github.com/Sidhu1504/my_app.git
ls my_app/
# index.html
sudo mv my_app/index.html /var/www/html/
```

git clone — my_app repo cloned, contains index.html

```
ubuntu@ip-172-31-39-53:~$ ls
ubuntu@ip-172-31-39-53:~$ git clone https://github.com/Sidhu1504/my_app.git
Cloning into 'my_app'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
ubuntu@ip-172-31-39-53:~$ ls
my_app
ubuntu@ip-172-31-39-53:~$ ls my_app/
index.html
ubuntu@ip-172-31-39-53:~$
```

Moving index.html to /var/www/html/ — then testing Nginx config (note: use sudo nginx -t)

```
ubuntu@ip-172-31-39-53:~$ sudo mv my_app/index.html /var/www/html/
ubuntu@ip-172-31-39-53:~$ nginx -t
2026/03/18 03:43:26 [warn] 1902#1902: the "user" directive makes sense only if the master process runs with super-user privileges, ignored in /etc/nginx/nginx.conf:1
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
2026/03/18 03:43:26 [emerg] 1902#1902: open() "/run/nginx.pid" failed (13: Permission denied)
nginx: configuration file /etc/nginx/nginx.conf test failed
ubuntu@ip-172-31-39-53:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

⚠ **Permission Note:** Running nginx -t without sudo gives a 'Permission denied' warning for /run/nginx.pid. Always use sudo nginx -t for correct results.

Step 5 — Start and Enable Nginx

Start Nginx and configure it to auto-start on every server reboot:

```
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx
```

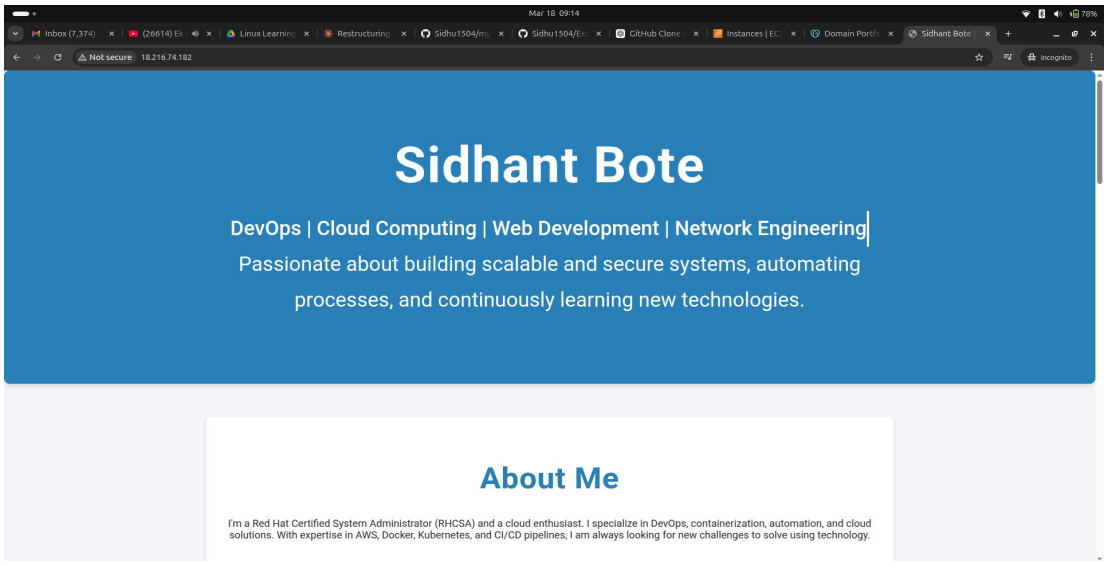
Nginx service: active (running) — enabled for auto-start, 2 worker processes, using 2.4 MB RAM

```
ubuntu@ip-172-31-39-53:~$ sudo systemctl start nginx
ubuntu@ip-172-31-39-53:~$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
ubuntu@ip-172-31-39-53:~$
ubuntu@ip-172-31-39-53:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-03-18 03:40:45 UTC; 3min 24s ago
       Docs: man:nginx(8)
   Main PID: 1794 (nginx)
      Tasks: 3 (limit: 2200)
     Memory: 2.4M (peak: 5.1M)
        CPU: 30ms
     CGroup: /system.slice/nginx.service
             ├─1794 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─1796 "nginx: worker process"
             └─1797 "nginx: worker process"

Mar 18 03:40:45 ip-172-31-39-53 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Mar 18 03:40:45 ip-172-31-39-53 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-39-53:~$
```

Step 6 — Test Website via IP

Before setting up DNS, verify the site loads via the raw public IP in your browser:

Portfolio site loading at http://18.216.74.182 — site working but no domain/HTTPS yet



What You See: The browser shows 'Not secure' and the IP in the address bar — this is expected. DNS and SSL come in the next steps.

Step 7 — Configure Nginx for Your Domain

Edit the Nginx default site config to recognise your domain:

```
sudo vim /etc/nginx/sites-available/default
```

Nginx server block configured for sidhubote.in and www.sidhubote.in

```
server {
    listen 80;
    server_name sidhubote.in www.sidhubote.in;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
~
```

The key directives:

- listen 80 — accept HTTP traffic on port 80
- server_name sidhubote.in www.sidhubote.in — respond only to requests for your domain
- root /var/www/html — directory where your index.html lives
- try_files $uri $uri/ =404 — serve files or return 404 if not found

```
sudo nginx -t     # Test config is valid
sudo systemctl reload nginx    # Apply changes without downtime
```

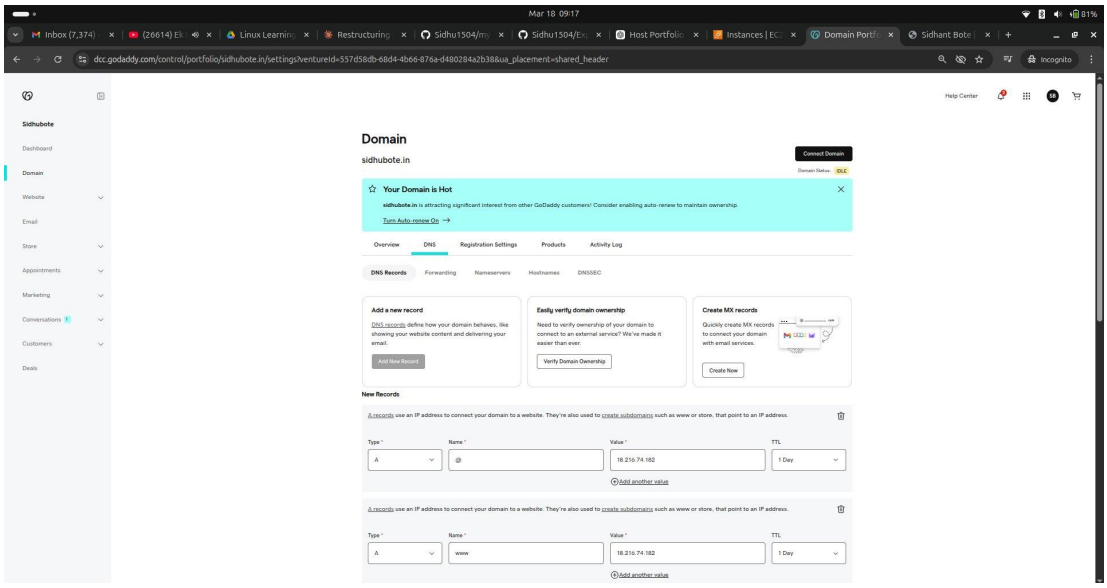
Config test passes, Nginx reloaded successfully, still active (running)

```
ubuntu@ip-172-31-39-53:~$ sudo vim /etc/nginx/sites-available/default
ubuntu@ip-172-31-39-53:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-39-53:~$ sudo systemctl reload nginx
ubuntu@ip-172-31-39-53:~$
ubuntu@ip-172-31-39-53:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-03-18 03:40:45 UTC; 11min ago
       Docs: man:nginx(8)
    Process: 2146 ExecReload=/usr/sbin/nginx -g daemon on; master_process on; -s reload (code=exited, status=0/SUCCESS)
   Main PID: 1794 (nginx)
      Tasks: 3 (limit: 2200)
     Memory: 2.6M (peak: 5.1M)
        CPU: 45ms
     CGroup: /system.slice/nginx.service
             ├─1794 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─2148 "nginx: worker process"
             └─2149 "nginx: worker process"

Mar 18 03:40:45 ip-172-31-39-53 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Mar 18 03:40:45 ip-172-31-39-53 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
Mar 18 03:52:12 ip-172-31-39-53 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server...
Mar 18 03:52:12 ip-172-31-39-53 nginx[2146]: 2026/03/18 03:52:12 [notice] 2146#2146: signal process started
Mar 18 03:52:12 ip-172-31-39-53 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-39-53:~$
```

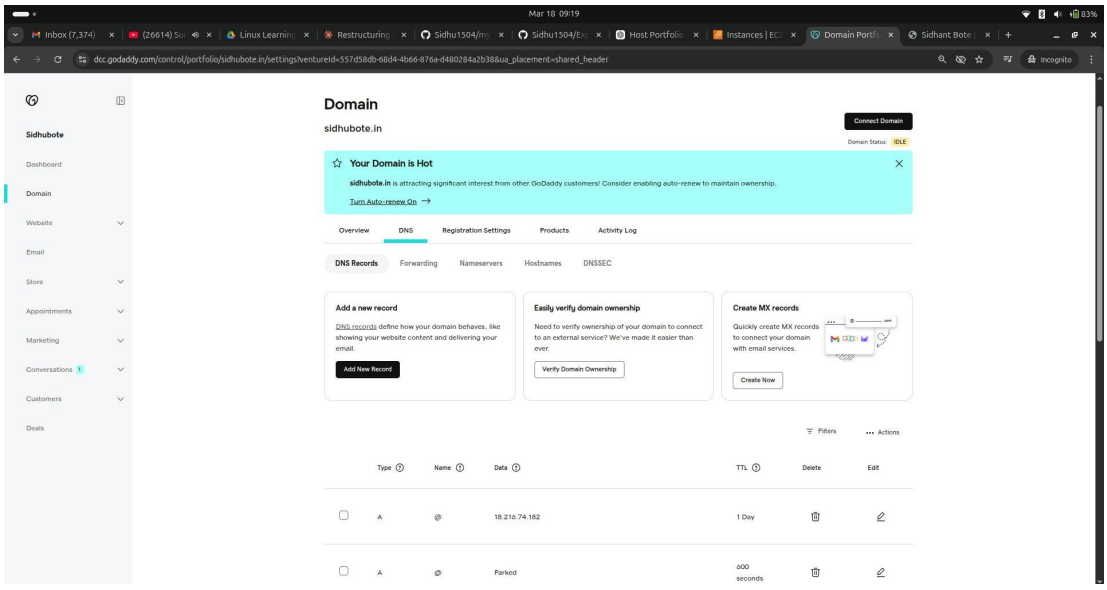
Step 8 — Point Your Domain to EC2 via GoDaddy DNS

Log into GoDaddy → Domain → DNS tab → Add New Records. Create two A records:

GoDaddy DNS — adding A records: @ → 18.216.74.182 and www → 18.216.74.182



GoDaddy DNS Records saved — A record @ → 18.216.74.182 confirmed with 1 Day TTL



⏳ **Propagation:** DNS changes can take up to 48 hours globally. In practice, usually under 1 hour. You can check status at dnschecker.org.

Step 9 — Install Certbot

Certbot is the official Let's Encrypt client. The Nginx plugin automates SSL certificate installation and Nginx config changes:

```
sudo apt install certbot python3-certbot-nginx -y
```

Certbot installation — 9 packages installed including python3-certbot-nginx plugin

```
ubuntu@ip-172-31-39-53:~$ sudo apt install certbot python3-certbot-nginx -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-acme python3-certbot python3-configargparse python3-icu python3-josepy python3-parsedatetime python3-rfc3339
Suggested packages:
  python-certbot-doc python3-certbot-apache python-acme-doc python-certbot-nginx-doc
The following NEW packages will be installed:
  certbot python3-acme python3-certbot python3-certbot-nginx python3-configargparse python3-icu python3-josepy python3-parsedatetime python3-rfc3339
0 upgraded, 9 newly installed, 0 to remove and 11 not upgraded.
Need to get 1097 kB of archives.
After this operation, 5699 kB of additional disk space will be used.
Get:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-josepy all 1.14.0-1 [22.1 kB]
Get:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-rfc3339 all 1.1-4 [6744 B]
Get:3 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-acme all 2.9.0-1 [48.5 kB]
Get:4 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-configargparse all 1.7-1 [31.7 kB]
Get:5 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-parsedatetime all 2.6-3 [32.8 kB]
Get:6 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-certbot all 2.9.0-1 [267 kB]
Get:7 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 certbot all 2.9.0-1 [89.2 kB]
Get:8 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-certbot-nginx all 2.9.0-1 [66.0 kB]
Get:9 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/main amd64 python3-icu amd64 2.12-1build2 [534 kB]
Fetched 1097 kB in 0s (13.5 MB/s)
Preconfiguring packages ...
Selecting previously unselected package python3-josepy.
(Reading database ... 72614 files and directories currently installed.)
Preparing to unpack .../0-python3-josepy_1.14.0-1_all.deb ...
Unpacking python3-josepy (1.14.0-1) ...
Selecting previously unselected package python3-rfc3339.
Preparing to unpack .../1-python3-rfc3339_1.1-4_all.deb ...
```

Step 10 — Get the SSL Certificate

Run Certbot with the --nginx flag. It will obtain the certificate AND automatically update your Nginx config:

```
sudo certbot --nginx -d sidhubote.in
```

Certbot success — certificate saved, HTTPS enabled on https://sidhubote.in, expires 2026-06-16

```
ubuntu@ip-172-31-39-53:~$ sudo certbot --nginx -d sidhubote.in
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Enter email address (used for urgent renewal and security notices)
 (Enter 'c' to cancel): botesidhant@gmail.com

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Please read the Terms of Service at
https://letsencrypt.org/documents/LE-SA-v1.6-August-18-2025.pdf. You must agree
in order to register with the ACME server. Do you agree?
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(Y)es/(N)o: y

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Would you be willing, once your first certificate is successfully issued, to
share your email address with the Electronic Frontier Foundation, a founding
partner of the Let's Encrypt project and the non-profit organization that
develops Certbot? We'd like to send you email about our work encrypting the web,
EFF news, campaigns, and ways to support digital freedom.
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(Y)es/(N)o: y
Account registered.
Requesting a certificate for sidhubote.in

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/sidhubote.in/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/sidhubote.in/privkey.pem
This certificate expires on 2026-06-16.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

Deploying certificate
Successfully deployed certificate for sidhubote.in to /etc/nginx/sites-enabled/default
Congratulations! You have successfully enabled HTTPS on https://sidhubote.in

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
If you like Certbot, please consider supporting our work by:
 * Donating to ISRG / Let's Encrypt:   https://letsencrypt.org/donate
 * Donating to EFF:                    https://eff.org/donate-le
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

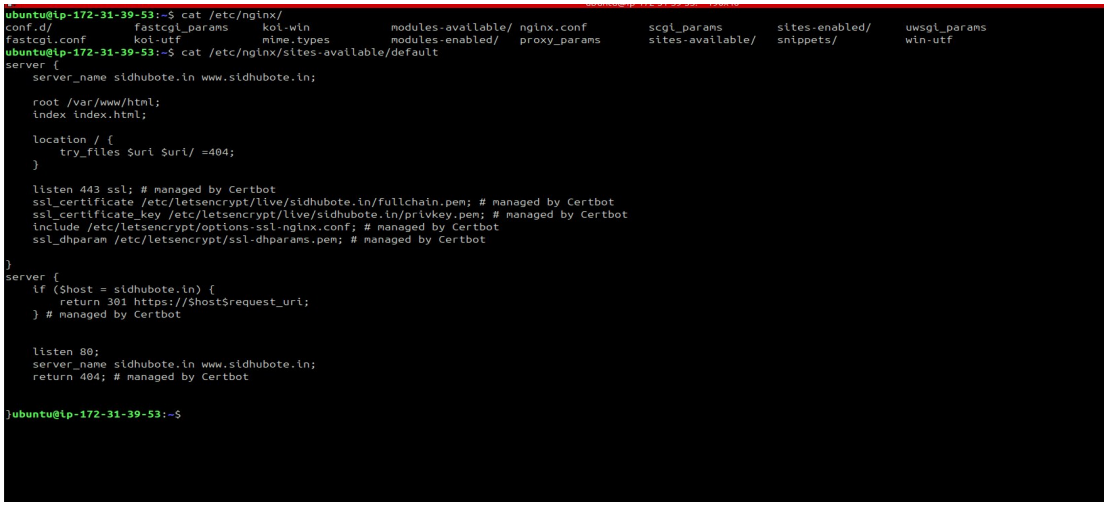
- Certificate stored at: /etc/letsencrypt/live/sidhubote.in/fullchain.pem
- Private key stored at: /etc/letsencrypt/live/sidhubote.in/privkey.pem
- Auto-renewal scheduled via systemd timer — renews when < 30 days remain
- Certbot automatically added SSL directives to your Nginx config

Step 11 — Review Final Nginx Config

After Certbot runs, check what it added to your Nginx config:

```
cat /etc/nginx/sites-available/default
```

Final Nginx config — Certbot added SSL cert paths, listen 443 ssl, and HTTP→HTTPS redirect block



```
ubuntu@ip-172-31-39-53:~$ cat /etc/nginx/
conf.d/            fastcgi_params     koi-win            modules-available/ nginx.conf         scgi_params        sites-enabled/     uwsgi_params
fastcgi.conf       koi-utf            mime.types         modules-enabled/   proxy_params       sites-available/   snippets/          win-utf
ubuntu@ip-172-31-39-53:~$ cat /etc/nginx/sites-available/default
server {
    server_name sidhubote.in www.sidhubote.in;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/sidhubote.in/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/sidhubote.in/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot

}
server {
    if ($host = sidhubote.in) {
        return 301 https://$host$request_uri;
    } # managed by Certbot


    listen 80;
    server_name sidhubote.in www.sidhubote.in;
    return 404; # managed by Certbot


}ubuntu@ip-172-31-39-53:~$
```

Certbot added three important things:

- listen 443 ssl with the certificate and key paths
- A second server block that listens on port 80 and redirects all HTTP traffic to HTTPS (301)
- include /etc/letsencrypt/options-ssl-nginx.conf — strong SSL settings

Step 12 — Verify HTTP → HTTPS Redirect

Test both HTTP and HTTPS from the command line with curl:

```
curl sidhubote.in
# Returns: 301 Moved Permanently (redirects to https://)

curl https://sidhubote.in
# Returns: the actual HTML of your portfolio page
```

curl results — http:// returns 301 redirect, https:// returns full page HTML

```
ubuntu@ip-172-31-39-53:~$ curl sidhubote.in
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
ubuntu@ip-172-31-39-53:~$
ubuntu@ip-172-31-39-53:~$ curl https://sidhubote.in
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Sidhant Bote | Portfolio</title>
  <link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;500;700&display=swap" rel="stylesheet">
  <style>
    /* Global styles */
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: 'Roboto', sans-serif;
    }

    body, html {
      height: 100%;
      background-color: #f4f4f9;
      color: #333;
      overflow-x: hidden;
      font-size: 18px; /* Default font size increased for better readability */
    }

    html {
      scroll-behavior: smooth;
    }

    /* Container and Layout */
```

Step 13 — Verify DNS Propagation

Confirm your domain resolves to the correct IP using dig:

```
dig +short sidhubote.in
```

dig +short sidhubote.in → 18.216.74.182 — DNS is propagated correctly

```
ubuntu@ip-172-31-39-53:~$ dig +short sidhubote.in
18.216.74.182
ubuntu@ip-172-31-39-53:~$
```

Step 14 — Test Auto-Renewal

Let's Encrypt certificates expire every 90 days. Test that auto-renewal works without actually renewing:

```
sudo certbot renew --dry-run
```

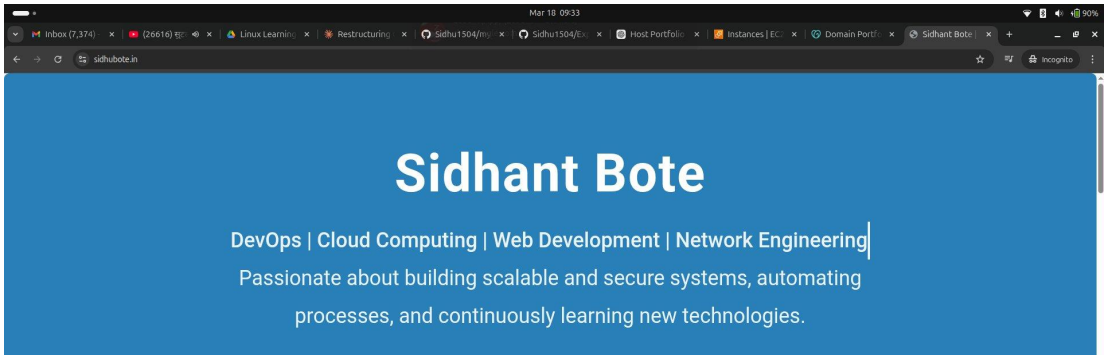
Certbot dry-run success — simulated renewal passed, auto-renewal is working correctly

```
ubuntu@ip-172-31-39-53:~$ sudo certbot renew --dry-run
Saving debug log to /var/log/letsencrypt/letsencrypt.log

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Processing /etc/letsencrypt/renewal/sidhubote.in.conf
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Account registered.
Simulating renewal of an existing certificate for sidhubote.in

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Congratulations, all simulated renewals succeeded:
  /etc/letsencrypt/live/sidhubote.in/fullchain.pem (success)
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

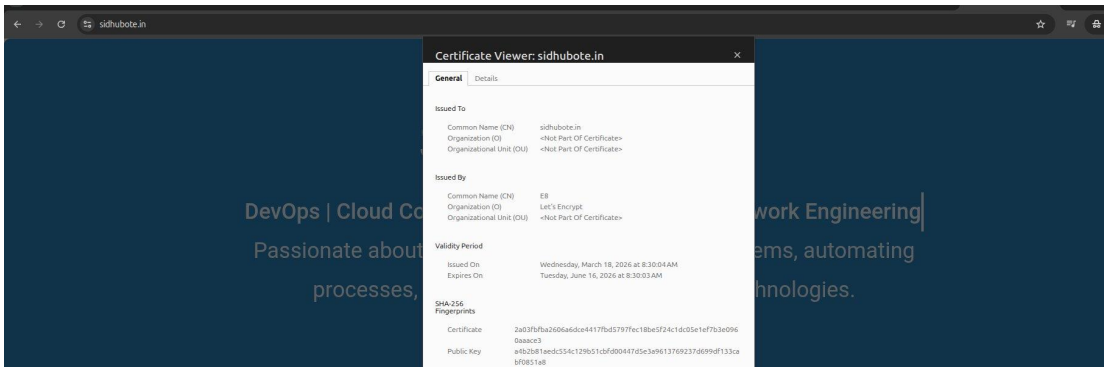
Step 15 — Final Result: Live HTTPS Site

Open https://sidhubote.in in your browser — the site is live with a valid SSL certificate:

sidhubote.in live with HTTPS — padlock visible, URL shows sidhubote.in (no 'Not secure')



Certificate Viewer — Issued to: sidhubote.in • Issued by: Let's Encrypt (E8) • Valid until: June 16, 2026



✅ **Success:** Portfolio site live at https://sidhubote.in — EC2 + Nginx + GoDaddy DNS + Let's Encrypt SSL — fully working end to end.

Chapter 3 — Quick Reference

dig Commands

Command	Purpose
dig sidhubote.in	Look up A record (IPv4 address)
dig sidhubote.in MX	Find mail servers
dig sidhubote.in NS	Find name servers
dig sidhubote.in TXT	Read TXT records (SPF, DKIM, verification)
dig +short sidhubote.in	Just the IP — no headers
dig @8.8.8.8 sidhubote.in	Query Google's DNS (test propagation)
dig +trace sidhubote.in	Full resolution from root → TLD → authoritative NS
dig -x 18.216.74.182	Reverse DNS: IP → hostname

Certbot Commands

Command	Purpose
sudo certbot --nginx -d example.com	Get cert + auto-configure Nginx
sudo certbot --nginx -d example.com -d www.example.com	Cover root + www in one cert
sudo certbot renew --dry-run	Test auto-renewal (safe — doesn't actually renew)
sudo certbot renew	Force manual renewal
sudo certbot certificates	List all installed certificates and expiry dates

Nginx Cheat Sheet

Command	Purpose
sudo nginx -t	Test config syntax (always do this before reloading)
sudo systemctl reload nginx	Apply config changes without downtime
sudo systemctl restart nginx	Full restart (brief downtime)
sudo systemctl status nginx	Check if Nginx is running
sudo systemctl enable nginx	Auto-start Nginx on reboot
cat /etc/nginx/sites-available/default	View current Nginx site config

DNS Record Types

Record	Maps	Example
A	Domain → IPv4	sidhubote.in → 18.216.74.182
AAAA	Domain → IPv6	sidhubote.in → 2606:...
CNAME	Name → another name (alias)	www → sidhubote.in
MX	Domain → mail server	priority 10 mail.example.com
TXT	Domain → text data	v=spf1 include:_spf.google.com ~all
NS	Domain → authoritative name servers	ns1.cloudflare.com
PTR	IP → hostname (reverse DNS)	18.216.74.182 → sidhubote.in
CAA	Domain → permitted SSL certificate issuers	0 issue 'letsencrypt.org'

Key Concepts Glossary

Term	Meaning
DNS	Domain Name System — translates names to IPs
A Record	The core record linking a domain to an IPv4 address
TTL	Time to Live — how long (seconds) a DNS answer is cached
DNS Propagation	Time for changes to spread to all resolvers worldwide (up to 48h)
Authoritative NS	The name server that holds the real records for your domain
Recursive Resolver	DNS server that chases down the answer on your behalf (your ISP/Google)
CA (Certificate Authority)	Organisation that issues SSL/TLS certs — e.g., Let's Encrypt
DV Certificate	Domain Validation — proves you own the domain (cheapest/free)
ACME	Protocol for automated cert issuance — what Certbot uses with Let's Encrypt
Certbot	Official Let's Encrypt client — automates getting and renewing SSL certs
TLS Handshake	Process where browser and server agree on encryption and verify the cert
301 Redirect	Permanent redirect — HTTP → HTTPS uses this to enforce secure connections